F1 Tire Strategy: Optimization Techniques & Review Prep

Summary

Our final answer is **pit on lap 34**: Differential Evolution raises the win probability from **78.00%** (the old grid's best, lap 42) to **99.99%**. The project asks one question: *when should our driver pit to finish ahead of the rival car?* We answer it with real Bahrain GP 2023 data, five tire-degradation models, a Monte Carlo race simulator and an optimization algorithm.

Number to remember	Value
Optimal pit lap (Differential Evolution)	**Lap 34** (current lap 30, window 31–56)
Ensemble win probability at lap 34	**99.99%** (XGBoost 99.99%, GP 99.98%)
Old 4-point grid's best	Lap 42, **78.00%**
Improvement	**+21.99 percentage points**; +5.06 s (XGB) and +18.66 s (GP) extra median margin
DE effort	19 unique laps evaluated, 3 generations, about 1 s, 50,000 races per lap
Best RMSE	Random Forest, 0.648 s
Best strategy cost (SIS)	Gaussian Process, 2.15 s per stint
Data	564 clean laps, 19 drivers, 52 stints (FastF1, Bahrain 2023)

Work done up to Review 2

By Review 2 the full prediction-and-simulation pipeline was built and ended in a 4-point strategy grid (best: lap 42). The Differential Evolution optimizer is the new part added for this review.

1. **Data:** Bahrain GP 2023 race laps and weather loaded with FastF1.
2. **Cleaning:** kept accurate laps only; removed safety-car laps, pit in/out laps, dirty-air laps (less than 2 s behind another car) and ±2.5σ outliers. Result: 564 laps, 52 stints of 3+ laps.
3. **Fuel correction (physics):** a lighter car is faster, so we removed fuel effect using 0.055 s per kg, about 0.106 s per lap. What remains is tire wear.
4. **Target:** `LapTimeDelta` = fuel-corrected lap time minus that stint's fastest lap, i.e. the pure time lost to tire aging.
5. **Features:** tire age, age², age³, compound, track temperature, air temperature, lap number.
6. **Leakage-safe split:** whole stints held out (GroupShuffleSplit by stint): 452 train laps, 112 test laps.

7. **Five models:** Linear (baseline), Polynomial degree 2, XGBoost with a custom asymmetric loss, Random Forest, Gaussian Process.
8. **Strategy-aware metrics:** besides RMSE/MAE/R^2, we defined TW-RMSE (errors on old tires weighted up to 4×) and SIS (seconds lost per stint by trusting an over-optimistic model).
9. **Monte Carlo race simulator:** simulates the last 27 laps against a rival 2.1 s ahead who pits on lap 45, with 22 s pit loss and a 15% safety-car chance per 10 laps. Two engines: XGBoost (fixed noise) and GP (noise that grows with tire age).
10. **Strategy grid (Review 2 result):** 10,000 races each for pit laps 42, 45, 48, 51. Best was lap 42.
11. **Validation:** model drift of only 0.58 s over Perez's 20-lap stint; recommendation compared with real teams' pit stops (laps 35–50).
12. **Dashboard:** a Streamlit app showing all results, with a live what-if simulator.

Added after Review 2: a full-window Differential Evolution optimizer over laps 31–56, with benchmarks against grid search, golden-section and Newton–Raphson. It moved the answer from lap 42 (78.00%) to lap 34 (99.99%).

Optimization techniques used

The project optimizes at six levels, from model settings up to the race decision. The headline one is Differential Evolution, which chooses the pit lap.

#	Technique	What it optimizes	Result
1	Optuna (TPE, Bayesian search) + 5-fold CV	XGBoost and Random Forest settings	XGB: 196 trees, depth 3, learning rate 0.011. RF: 224 trees, depth 5
2	AIC / BIC / cross-validation	Polynomial degree (1–6)	Degree 2
3	Marginal-likelihood maximization	Gaussian Process kernel parameters	7 length scales + noise level learned
4	Asymmetric (cost-sensitive) loss	What XGBoost learns during training	Over-optimistic errors penalized 2.5×
5	Monte Carlo simulation (grid)	Pit lap, 4 candidates (Review 2)	Lap 42, 78.00%
6	**Differential Evolution** + common random numbers	**Pit lap, all 26 candidates**	**Lap 34, 99.99%**
+	Vectorization (computational)	Simulation speed	Hours to seconds

1. Optuna hyperparameter search (TPE)

What it does: finds the best model settings (number of trees, depth, learning rate, sampling rates, regularization).

How it works: Optuna's Tree-structured Parzen Estimator learns from earlier trials which settings tend to score well and samples new settings from those regions. It is smarter than grid search (too many combinations) and random search (wasteful). Each trial is scored by **5-fold cross-validation RMSE**: train on 4/5 of the data, test on the other 1/5, five times. XGBoost got 100 trials, Random Forest 30.

2. Polynomial degree selection (AIC / BIC / CV)

What it does: picks how curved the polynomial model should be.

How it works: degrees 1 to 6 were fitted and compared. AIC and BIC reward a good fit but penalize extra complexity (BIC more strongly). Cross-validation measures error on unseen data. Degree 2 was chosen as the simplest model at the BIC minimum.

3. Gaussian Process kernel optimization

What it does: tunes the GP's kernel (overall scale, one length scale per feature, noise level).

How it works: it maximizes the **log marginal likelihood**, i.e. how probable the observed data is under those parameters, using gradient-based optimization with 2 restarts. This balances fit against complexity automatically: features that don't help get very long length scales and are effectively switched off.

4. Asymmetric loss for XGBoost (cost-sensitive learning)

What it does: changes what "good" means during training, so the model avoids the expensive mistake.

How it works: if the model says the tire is faster than it really is (an optimistic error), the team stays out too long and loses the race. So that error gets 2.5× the weight of a pessimistic one. XGBoost only needs the gradient and Hessian of the loss:

$$g = \begin{cases} 2.5\,(\hat{y}-y) & \text{if } \hat{y} < y \\ 1.0\,(\hat{y}-y) & \text{otherwise} \end{cases} \qquad h = \begin{cases} 2.5 & \text{if } \hat{y} < y \\ 1.0 & \text{otherwise} \end{cases}$$

5. Monte Carlo strategy grid (Review 2)

What it does: estimates the win probability of a pit lap. There is no formula for it, so we simulate.

How it works: for each of 4 pit laps (42, 45, 48, 51), 10,000 races are simulated with random lap-time noise and random safety cars, and we count how often we finish ahead. This is simulation-based optimization (sample average approximation). Limitation: it only looks at 4 of the 26 possible laps.

6. Differential Evolution (final pit-lap optimizer)

What it does: searches **every** feasible pit lap (31–56) for the one with the highest win probability.

The objective is the average win probability of the two simulation engines, plus a tiny tie-breaker on the median finishing gap:

$$\max_{p \in \{31,\dots,56\}} \; J(p) = \tfrac{1}{2}\left[P_{XGB}(\text{win} \mid p) + P_{GP}(\text{win} \mid p)\right] + 10^{-6}\,\overline{\text{median gap}}(p)$$

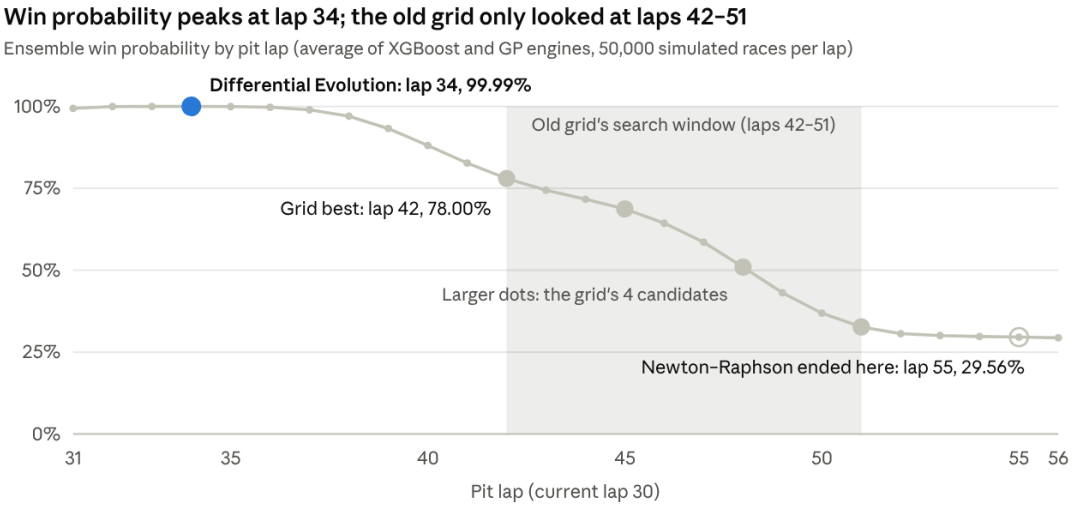
How it works (SciPy `differential_evolution`):

1. **Initialize:** a population of 12 random candidate pit laps across the window.

2. **Mutation:** for each candidate, build a mutant from the best one plus a scaled difference of two others: $v = best + F \times (x_1 - x_2)$, with F random in 0.5–1.0 (strategy `best1bin`).
3. **Crossover:** mix the mutant with the original (rate 0.7) and round to a whole lap.
4. **Selection:** simulate the trial lap (50,000 races per engine). It replaces the original only if it scores better.
5. **Repeat** until the population converges (tolerance 0.0001, at most 30 generations). Ours converged in **3 generations, 19 unique laps**.

Why it fits this problem: the objective is a noisy black box, integer-valued, not differentiable and possibly has more than one peak (the GP engine has a second local peak near lap 54). DE needs no derivative, handles integers and bounds, and searches globally.

Common random numbers: every candidate lap faces the *same* 50,000 random race scenarios (same safety cars, same noise), so differences come from the strategy, not luck. **Robustness:** DE seeds 42, 7 and 2026 all give lap 34; five fresh validation seeds × 20,000 races confirm it (99.98% vs 78.05%).



artifacts/optimization_landscape.csv · laps 31–56 · post-hoc check, not the optimizer itself

The grid only sampled the falling slope after lap 42, so it could never find the peak. DE searched the whole window and found lap 34 after evaluating 19 laps; this full scan just confirms it.

Bonus: computational optimization (vectorization)

Simulating naively needs about 2.16 million GP predictions (hours). Because a strategy's tire states are known in advance, we predict all 216 unique lap-states once, in one batch, and simulate all 10,000 races at once as NumPy arrays. That turns hours into seconds, which is why the dashboard's what-if simulator runs live.

Techniques we tried that didn't work

We implemented the classic methods as benchmarks. Each failed for a specific reason, and that is exactly why we chose Differential Evolution.

Technique	What happened	Why it failed
Grid search (pit laps 42, 45, 48, 51)	Picked lap 42 with 78.00% win	Tested only 4 of 26 possible laps, so it never saw the real peak at lap 34
Golden-section search, old bracket 42–51	Returned lap 42 (78.00%) after 7 evaluations	Assumes a single peak inside the bracket; it got stuck at the bracket edge, repeating the grid's bias
Golden-section search, full window 31–56	Found lap 34 in 9 evaluations	Worked here, but only because it was given the right bracket. It assumes one peak and can't guarantee the global best (the GP engine has a second peak near lap 54)
Newton–Raphson (quadratic surrogate, started at lap 42)	Jumped to lap 55 with only 29.56% win	Needs a smooth, differentiable function; ours is noisy and integer-valued. Its roots from different starts ranged from lap 5 to lap 148, outside the race
Polynomial regression (degree 2)	Worst model: RMSE 0.822 s, R^2 0.21, SIS 3.65 s	Too rigid for the tire "cliff"; it extrapolates badly at high tire ages
Optimizing one engine only	XGBoost-only picks lap 36, GP-only picks lap 33	Each bakes in one model's assumptions about uncertainty. We average both engines instead, which gives lap 34
Plain RMSE to pick the model	Random Forest wins on RMSE (0.648 s)	RMSE ignores *which* errors cost races. On SIS (strategy cost) the Gaussian Process is best (2.15 s), so RMSE alone would mislead

The lesson to say out loud: grid and golden-section depend on where you look, and Newton depends on smooth calculus. DE is the only one that is global, works with whole-number laps, needs no derivative and gave the same answer from every seed.

Likely teacher questions

These are the questions most likely to come up, with short answers either of us can give.

Q: What exactly are you optimizing? The pit lap p (an integer, 31–56) that maximizes our win probability against the rival, averaged over two simulation engines. Below that we also optimize model settings (Optuna), polynomial degree (AIC/BIC), GP kernel (marginal likelihood) and the training loss (asymmetric).

Q: Why Differential Evolution and not Newton–Raphson or golden-section? Win probability comes from simulation, so it is noisy, integer-valued and not differentiable: Newton can't work (it went to lap 55). Golden-section assumes one peak and got stuck at lap 42 with the old bracket. DE needs no derivative, handles integers and searches globally.

Q: It's only 26 laps, why not try them all? For one pit stop, brute force is cheap, and our full landscape plot confirms lap 34. DE's value is that the same method scales to two pit stops or choosing tire compounds, where trying every combination explodes.

Q: How does DE work, in one breath? Keep 12 candidate laps. Make new ones by adding the scaled difference of two candidates to the best one, mix, keep the new one only if it wins more often. Repeat until they agree. It took 3 generations.

Q: How do you know lap 34 isn't just luck? Common random numbers (every lap faces the same 50,000 races), three DE seeds all giving lap 34, and five fresh validation seeds (99.98% vs 78.05%).

Q: Why does the optimizer give the same numbers every run? It uses a fixed random seed (42) so results are reproducible. Changing the seed (`--seed 7`) changes the percentages slightly, but the answer stays lap 34.

Q: Why is the notebook's answer lap 33 but the dashboard's lap 34? The notebook's Optuna search has no fixed seed, so a fresh re-run picks slightly different XGBoost settings. The optimum moves by one lap and stays inside the 33–36 window that all our objective variants agree on, so the recommendation is robust.

Q: Why a custom asymmetric loss? An optimistic error (tire looks faster than it is) keeps the driver out too long and costs the race, so we penalize it 2.5×. XGBoost accepts any loss with a valid gradient and Hessian.

Q: Isn't RMSE 0.65 s poor? Normal lap-to-lap noise in F1 is about 0.3–0.7 s. We test on whole unseen stints, which is harder. What matters is the decision: drift was only 0.58 s over a 20-lap stint.

Q: How do you know the model didn't cheat? We hold out entire stints (GroupShuffleSplit), so no lap of a test stint is ever seen in training.

Q: Why do the XGBoost and GP engines disagree (99.64% vs 56.35% at lap 42)? They disagree about confidence, not direction. XGBoost uses one fixed noise level; the GP knows uncertainty grows as tires age, so its probabilities are closer to 50%. Averaging both protects us from one model's assumptions.

Q: Is lap 34 realistic? Real teams pitted between laps 35 and 50 (median 41). Lap 34 is an aggressive undercut, one lap before the earliest real stop, aimed at a rival who pits on lap 45.

Q: Limitations? One race only (Bahrain 2023); only 8 medium-tire laps; the rival follows a fixed rule (pits lap 45); a simple safety-car model; one-stop strategies only.

Presenting and preparing

Split the talk into two halves so each person owns one story end to end. Fill in the names.

Part	Covers	What to show	Presenter
1. Problem and data	Question, cleaning, fuel correction, leakage-safe split	Dashboard: Overview, Data & Method, EDA tabs	
2. Models	5 models, asymmetric loss,	Dashboard: Models tab	

Part	Covers	What to show	Presenter
	Optuna, RMSE vs SIS		
3. Simulation	Monte Carlo engines, Review 2 grid (lap 42)	Dashboard: Monte Carlo tab, move the Pit lap slider and run 10,000 simulations	
4. Optimization	DE, failed methods, validation, lap 34	Dashboard: Optimizer and Proof tabs; double-click `run_optimizer.bat` live	